

GraphQL Sequelize Application (React Assignment)

1. Project Overview and Purpose

Crate is a full-stack, monorepo-based subscription platform for trendy clothes and accessories. It offers a **monthly subscription service** and is designed to be a **developer-friendly, scalable, and modular** application across **web, mobile, and API layers**.

The purpose of Crate is to:

- **Provide a real-world, full-stack application** to help developers learn and implement advanced concepts like:
 - React + Redux for Web UI with SSR (Server Side Rendering)
 - React Native for Mobile Apps (Android & iOS)
 - GraphQL API with Node.js, Express, Sequelize (MySQL), and JWT Authentication
 - Scalable monorepo architecture
- **Enable seamless development and deployment** with Docker, database migrations, and data seeding.
- **Offer a production-grade architecture** for learning or extending into real business use cases.

Ideal for developers exploring:

- Full-stack JS development
- GraphQL APIs
- Mobile-first + SSR-friendly apps
- Scalable codebases with modular structure

2. Backend API Architecture

The **Crate backend** is built with a **modular and scalable Node.js architecture** using the following core technologies:

Tech Stack

- **Node.js:** Runtime environment.
- **Express.js:** Lightweight web framework for building REST and GraphQL APIs.
- **GraphQL:** Query language for API, enabling precise data fetching.
- **Sequelize ORM:** Object Relational Mapping for **MySQL** (configurable to PostgreSQL, SQLite, MSSQL).
- **JWT (JSON Web Token):** For user authentication and session management.

Folder Structure

Located in: code/api/src/

```
src/  
├─ config/          # Environment configs and database settings  
├─ migrations/      # Sequelize database migrations  
├─ modules/         # Main application logic organized by features  
                     (e.g., user, product, order)  
├─ seeders/         # Initial data population  
├─ setup/           # GraphQL setup, database initialization  
└─ index.js         # Entry point for API server
```

Key Features

- **GraphQL API** with clear schema definitions and field-level control.
- **Modular design:** Each domain (e.g., auth, product) is separated into its own module.
- **Authentication Middleware:** Secures private routes using JWTs.
- **File Upload Support:** GraphQL endpoint allows media uploads.
- **Database Layer:** Sequelize models define entities and relationships.
- **Migrations & Seeders:** Manage schema changes and sample data.

Running the Backend

- Start the API server:

```
cd code/api
npm start
```

- Access GraphQL Playground:

<http://localhost:8000>

This architecture ensures:

- **Clean separation of concerns**
- **Scalability** through modular code
- **Ease of testing, refactoring, and extending features**

Perfect for learning and deploying production-grade GraphQL APIs.

3. Database Configuration and Migrations

The Crate project uses **Sequelize** ORM to manage database interactions and schema through **migrations and seeders**.

Supported Databases

- Default: **MySQL**
- Also supports: PostgreSQL, SQLite, MSSQL (with minor config changes)

Key Files and Directories

Located in: code/api/src/

```
config/
└─ database.json      # Database credentials for different
```

environments (dev, test, prod)

migrations/

└─ *.js # Auto-generated Sequelize migration files for
schema changes

seeders/

└─ *.js # Seeder files to populate sample data into
tables

Configuring the Database

Edit the file:

code/api/src/config/database.json

Sample:

```
"development": {
  "username": "root",
  "password": null,
  "database": "crate",
  "host": "127.0.0.1",
  "port": 3307,
  "dialect": "mysql",
  "seederStorage": "sequelize"
},
"production": {
  "username": "root",
  "password": null,
  "database": "crate",
  "host": "127.0.0.1",
  "port": 3307,
  "dialect": "mysql",
  "seederStorage": "sequelize"
}
}
```

🔑 Tip: Use `.env` in `code/api/.env` for sensitive values like ports, JWT secrets.

Running Migrations and Seeders

Run the setup script:

```
cd code/api
npm run setup
```

Which does the following:

1. Installs dependencies
2. Runs migrations (`sequelize db:migrate`)
3. Runs seeders (`sequelize db:seed:all`)

Migration Lifecycle

- **Create a migration:**

```
npx sequelize-cli migration:generate --name create-user
```

- **Run migrations:**

```
npx sequelize-cli db:migrate
```

- **Undo migrations:**

```
npx sequelize-cli db:migrate:undo
```

This setup ensures:

- Structured and version-controlled DB schema
- Easy switching between environments

- Clean DB initialization during development or production setup

4. GraphQL API Implementation

The **Crate** project uses **GraphQL** for flexible data querying and mutation, integrated with **Apollo Server** for efficient API management.

Key Technologies

- **GraphQL**: Query language for APIs.
- **Apollo Server**: GraphQL server for Node.js.
- **Sequelize ORM**: Interacts with MySQL database.

Key Files

- **src/config/graphql.js**: GraphQL server and middleware setup.
- **src/modules/{module}/**: Contains GraphQL schemas, resolvers, and Sequelize models for each module (e.g., `user`, `product`).
- **Resolvers**: Handle fetching or mutating data from the database.
- **Sequelize models**: Define database schema and interact with MySQL.

Setup Process

1. **Server Setup**: Apollo Server is configured with type definitions and resolvers.
2. **Database Integration**: Resolvers use **Sequelize** to interact with the MySQL database.
3. **JWT Authentication**: JWT tokens are validated in the context of GraphQL to authorize user actions.

Example Queries/Mutations

- **Query**: Fetch a user by ID.
- **Mutation**: Create a new user.

This implementation ensures a scalable and modular architecture, with easy integration between the frontend, backend, and database.

5. Environment and Setup (/api/src/setup)

The **environment setup** for the **Crate** project is essential for configuring and running the backend API smoothly. Here's how to set it up:

Key Setup Files in /api/src/setup

- **database.js**: Configures the database connection (MySQL, PostgreSQL, etc.) using **Sequelize**.
- **graphqlSetup.js**: Initializes and configures the Apollo Server for GraphQL, linking it to the schema and resolvers.
- **.env**: Contains environment variables for configuration, such as port numbers, database credentials, and API keys.

Steps to Setup

1. Clone the Repo

```
git clone git@github.com:atulmy/crate.git crate
cd crate
```

2. Install Dependencies

Navigate to the /api directory and install the necessary Node.js packages:

```
cd api
npm install
```

3. Configure Database

Modify the **database.json** file in **/api/src/config/** to provide correct database credentials:

```
"username": "root",  
  "password": null,  
  "database": "crate",  
  "host": "127.0.0.1",  
  "port": 3307,  
  "dialect": "mysql",  
  "seederStorage": "sequelize"
```

4. **Environment Variables** Set up the `.env` file in `/api/` with configurations like port and database URL:

```
PORT=8000
```

```
DATABASE_URL=mysql://root:password@localhost:3306/crate_db
```

5. **Database Migrations and Seeding** Run the setup scripts to initialize the database schema and seed initial data:

```
npm run setup
```

Running the Application

- **API:** To start the API server:

```
npm start
```

The GraphQL server will be available at <http://localhost:8000>.

This setup ensures that all necessary configurations for the database, API, and environment are in place for smooth development and deployment.

6. Mobile Application Structure (/mobile)

The **mobile** folder in the **Crate** project contains the code for the mobile applications (Android and iOS) built using **React Native**.

Key Directories and Files in /mobile

1. **assets/**: Contains static assets such as images, icons, and fonts used in the mobile app.
2. **src/**: Main source code for the mobile application.
 - a. **modules/**: Contains feature-specific modules such as user authentication, product listing, and order management.
 - b. **setup/**: Contains configuration files like API URL setup and other environment-specific settings.
 - c. **ui/**: Contains reusable UI components (buttons, inputs, cards) that can be used across different screens.
 - d. **index.js**: The entry point for the React Native app that sets up the main application component.
3. **package.json**: Lists dependencies and scripts required for building and running the mobile app.
4. **ios/**: Contains iOS-specific configurations and native code for building the app for iOS devices.

Key Files and Their Purpose

- **src/setup/config.json**: Configuration file where the API URL and other environment-specific variables are set. For example:

```
{  
  "apiUrl": "http://localhost:8000"  
}
```

Modify this to point to the correct API endpoint.

- **src/modules/auth/**: Manages user authentication, including login, registration, and token management using JWT.
- **src/ui/**: Contains reusable components like `Button`, `InputField`, and `Card` that make up the mobile UI.

Setting Up the Mobile App

1. Install Dependencies

Navigate to the `mobile` directory and install required dependencies:

```
cd mobile
npm install
```

2. Install iOS Dependencies (For macOS users)

If you're building for iOS, install the CocoaPods dependencies:

```
cd ios
pod install
```

3. Running the App

a. For iOS:

```
npx react-native run-ios
```

b. For Android:

```
npx react-native run-android
```

Key Features

- **Cross-Platform:** Supports both Android and iOS using React Native.
- **Responsive UI:** The app is designed to adapt to different screen sizes and orientations.
- **State Management:** Uses **Redux** for managing global state across the app.
- **Authentication:** Handles JWT-based authentication for secure user sessions.

This structure enables efficient mobile development and provides a clear separation of concerns, making it easy to scale and manage.

7. Web Application Architecture (/web)

The **web** folder in the **Crate** project contains the code for the web application, built with **React**, **Redux**, and **Server-Side Rendering (SSR)** for better performance and SEO.

Key Directories and Files in /web

1. **public/**: Contains static assets such as images, icons, and the HTML template (`index.html`).
2. **src/**: Main source code for the React app.
 - a. **modules/**: Core features and logic (e.g., `auth`, `product`, `user`).
 - b. **setup/**: Configuration files (API URLs, routing, etc.).
 - c. **ui/**: Reusable UI components like buttons, cards, and inputs.
 - d. **index.js**: Entry point where the app is initialized.
3. **storybook/**: Contains Storybook setup for visualizing and testing UI components in isolation.
4. **server.js**: Sets up the SSR using **Express** for rendering React components on the server.

Key Features of the Web Application

- **Server-Side Rendering (SSR)**: The app uses SSR for faster page loads and better SEO. React components are rendered on the server before being sent to the client.
- **React and Redux**: Used for UI building and state management across the app.
- **GraphQL Integration**: The web app communicates with the backend using GraphQL for efficient data fetching.

Setup Process

1. Install Dependencies

Navigate to `/web` and install packages:

```
cd web
npm install
```

2. Configure API URL

Modify the `.env` file for the correct API URL and port:

```
REACT_APP_API_URL=http://localhost:8000
```

3. Run Development Server

Start the app in development mode:

```
npm start
```

The app will run on <http://localhost:3000>.

Production Setup

1. Build the App:

```
npm run build
```

2. Run in Production:

```
npm run start:prod
```

This architecture allows for a scalable, maintainable, and SEO-friendly web application, leveraging modern practices like SSR and React/Redux.

8. Storybook Integration (Web UI Documentation)

Storybook is used in the **Crate** project for **UI component documentation** and **visual testing**. It allows developers to create a library of reusable components, test them in isolation, and demonstrate their usage.

Storybook Structure in /web

1. **storybook/**: This directory contains the configuration for Storybook, enabling the development and preview of UI components in isolation.
2. **Component Stories**: Storybook "stories" are simple JavaScript functions that return UI components with different props and states, allowing you to see how each component behaves.
3. **UI Components**: Components located in the **src/ui/** folder are integrated into Storybook. Each UI component (e.g., buttons, cards, inputs) has an associated "story" file that showcases different states and use cases.

Key Steps for Storybook Integration

1. Install Storybook

If it's not already installed, you can set up Storybook by running:

```
npx sb init
```

2. Add Stories

For each UI component, create a `.stories.js` file. A simple example could be for a `Button` component:

```
import React from 'react';
import Button from './Button';

export default {
  title: 'Button',
  component: Button,
};

export const Primary = () => <Button primary>Primary Button</Button>;
export const Secondary = () => <Button>Secondary Button</Button>;
```

3. Run Storybook

Start the Storybook server to view and interact with your components:

```
npm run storybook
```

The Storybook UI will be available at <http://localhost:6006>.

Benefits of Storybook

- **Isolated Development:** UI components can be developed and tested independently, which speeds up development and testing processes.
- **Component Library:** Storybook acts as a living style guide for your components, providing documentation for how each component should behave with different states.
- **UI Testing:** Components can be visually tested in multiple states without needing to run the full application.

By integrating **Storybook** into the Crate web application, you can ensure that UI components are well-documented, easily reusable, and tested in isolation, providing an improved developer experience.

9. Project Execution & Deployment

Deploying and executing the **Crate** project involves multiple steps, including setting up the development environment, building the project, and deploying the application for production. Below is a guide for executing and deploying both the **API**, **Web**, and **Mobile** applications.

1. Development Environment Setup

Before deploying, ensure all prerequisites are met for both **API**, **Web**, and **Mobile**:

- **Node.js** installed.
- **MySQL** (or other database like PostgreSQL) set up and running.
- **React Native dependencies** installed for mobile development.

2. Project Setup

1. Clone the Repository

First, clone the repository to your local machine:

```
git clone git@github.com:atulmy/crate.git crate
cd crate
```

2. Configure Environment Variables

- a. **API:** Modify the `/api/.env` and `/api/src/config/database.json` to configure database credentials and API port.
 - b. **Web:** Modify `/web/.env` to set the correct API URL and port.
 - c. **Mobile:** Modify `/mobile/src/setup/config.json` for API URL (you can use `ifconfig` to get your local IP address).
3. **Install Dependencies** Install all dependencies for **API**, **Web**, and **Mobile** applications.
- a. For **API**:

```
cd api
npm install
npm run setup
```

- b. For **Web**:

```
cd web
npm install
```

- c. For **Mobile**:

```
cd mobile
npm install
cd ios && pod install && cd ..
```

3. Running the Development Environment

1. Run the API

Start the API server in development mode:

```
cd api  
npm start
```

The API will be accessible at <http://localhost:8000>.

2. Run the Web Application

Start the React web application in development mode:

```
cd web  
npm start
```

The web application will be accessible at <http://localhost:3000>.

3. Run the Mobile Application

To run on **iOS**:

```
cd mobile  
npx react-native run-ios
```

To run on **Android**:

```
npx react-native run-android
```

4. Production Build

1. **Build API for Production** To build the API for production:

```
cd api  
npm run build  
npm run start:prod
```

This creates an optimized build and runs the server.

2. **Build Web Application for Production** To build the React web application for production:

```
cd web
npm run build
```

The optimized build will be created in the `build` directory. You can serve it using a static server or deploy it to a cloud service (e.g., AWS, Netlify).

3. **Build Mobile Application for Production** To build the React Native mobile app for production:

```
cd mobile
npx react-native run-ios --configuration Release # For iOS
npx react-native run-android --variant=release # For Android
```

5. Deployment Options

Web Deployment

- You can deploy the web app using services like **Netlify**, **Vercel**, or **AWS S3**.
- For example, to deploy on **Netlify**:
 - Push the `build` folder to your GitHub repository.
 - Link the repository to Netlify.
 - Netlify will automatically deploy the static files.

API Deployment

- **Heroku**, **AWS EC2**, or **DigitalOcean** can be used to deploy the backend API.
 - For **Heroku**:
 - Create a Heroku app.
 - Set environment variables (`DATABASE_URL`, etc.).
 - Push the code to Heroku:

```
git push heroku master
```

Mobile App Deployment

- For **iOS**, submit the build to the **App Store** using **Xcode**.
- For **Android**, submit the APK to the **Google Play Store** using **Android Studio**.

6. Continuous Integration / Continuous Deployment (CI/CD)

To streamline deployment, you can set up **CI/CD pipelines** using tools like **GitHub Actions**, **Travis CI**, or **CircleCI**. This allows automated testing, building, and deployment of your application to production.

7. Monitoring and Maintenance

- Use monitoring tools like **New Relic** or **Prometheus** for tracking server health and API performance.
- Regularly update dependencies and monitor the app for bugs, security vulnerabilities, and other performance issues.

By following these steps, you will be able to set up, execute, and deploy the **Crate** project both locally and in production environments. This allows you to effectively manage all aspects of the web, API, and mobile applications.

Sure! Here's an explanation regarding the GitHub repository [GraphQL Sequelize Application](#):

GitHub Repository: GraphQL Sequelize Application

The **GraphQL Sequelize Application** repository is a full-stack application designed to demonstrate the integration of **GraphQL**, **Sequelize**, and **MySQL** for building modern web applications. This project features a backend API, a web frontend, and a mobile app, all organized within a well-structured monorepo architecture. The goal of this project is to provide developers with a robust template to build scalable and flexible applications with modern technologies.

Key Features

- **GraphQL API:** The backend is powered by GraphQL, providing a flexible and efficient API for querying and mutating data.
- **Sequelize ORM:** Sequelize is used for interacting with a MySQL database in an Object-Relational Mapping (ORM) manner, making it easy to handle database operations.
- **JWT Authentication:** Secure authentication is implemented using JSON Web Tokens (JWT) for user authentication in the GraphQL API.
- **Frontend Web Application:** The web frontend is built with **React** and **Redux** for state management, making the application highly interactive and responsive.
- **Mobile Application:** A mobile app is developed using **React Native**, providing both Android and iOS support.
- **Server-Side Rendering (SSR):** The web app supports SSR, making it SEO-friendly and improving initial page load performance.

Repository Structure

The repository follows a **monorepo** structure, with separate directories for each part of the application:

- **/api:** Contains the backend API implementation using GraphQL and Sequelize.
- **/web:** Holds the web application built with React and Redux.
- **/mobile:** Contains the React Native codebase for the mobile application.
- **/storybook:** Documentation for UI components using **Storybook**.

Technologies Used

- **Node.js:** The backend is built using Node.js for handling asynchronous requests and building the GraphQL API.
- **GraphQL:** A modern alternative to REST APIs that allows clients to request only the data they need.
- **Sequelize (MySQL):** Sequelize is used as an ORM to interact with the MySQL database, allowing for easy migrations, querying, and relationships.
- **JWT:** JSON Web Tokens are used for secure user authentication.
- **React & Redux:** The web frontend is built using React for UI rendering and Redux for state management.
- **React Native:** A framework for building mobile apps using React, with support for both Android and iOS.
- **Webpack & Babel:** Tools for bundling and transpiling the code to ensure compatibility with modern JavaScript standards.

- **Docker:** Used to containerize the application for easy deployment.

Setup and Installation

To get the project up and running locally, follow these steps:

1. **Clone the Repository:**

```
git clone https://github.com/yedlashivakumar/GraphQL-Sequelize-Application.git
```

2. **Install Dependencies:**

- a. For backend (/api), run `npm install`.
- b. For web (/web), run `npm install`.
- c. For mobile (/mobile), run `npm install` and for iOS, run `pod install` in the /mobile/ios directory.

3. **Run the Application:**

- a. Start the backend: `npm start` inside the /api folder.
- b. Start the web app: `npm start` inside the /web folder.
- c. For mobile: `npx react-native run-ios` for iOS or `npx react-native run-android` for Android.

Contributions

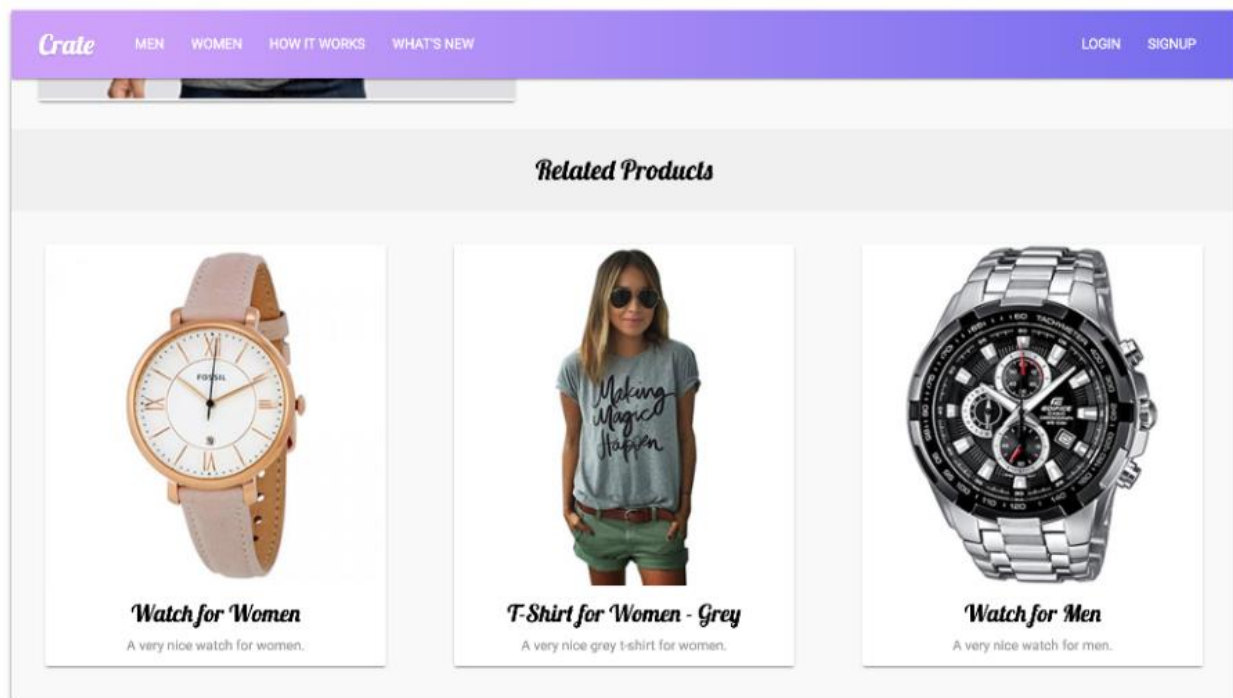
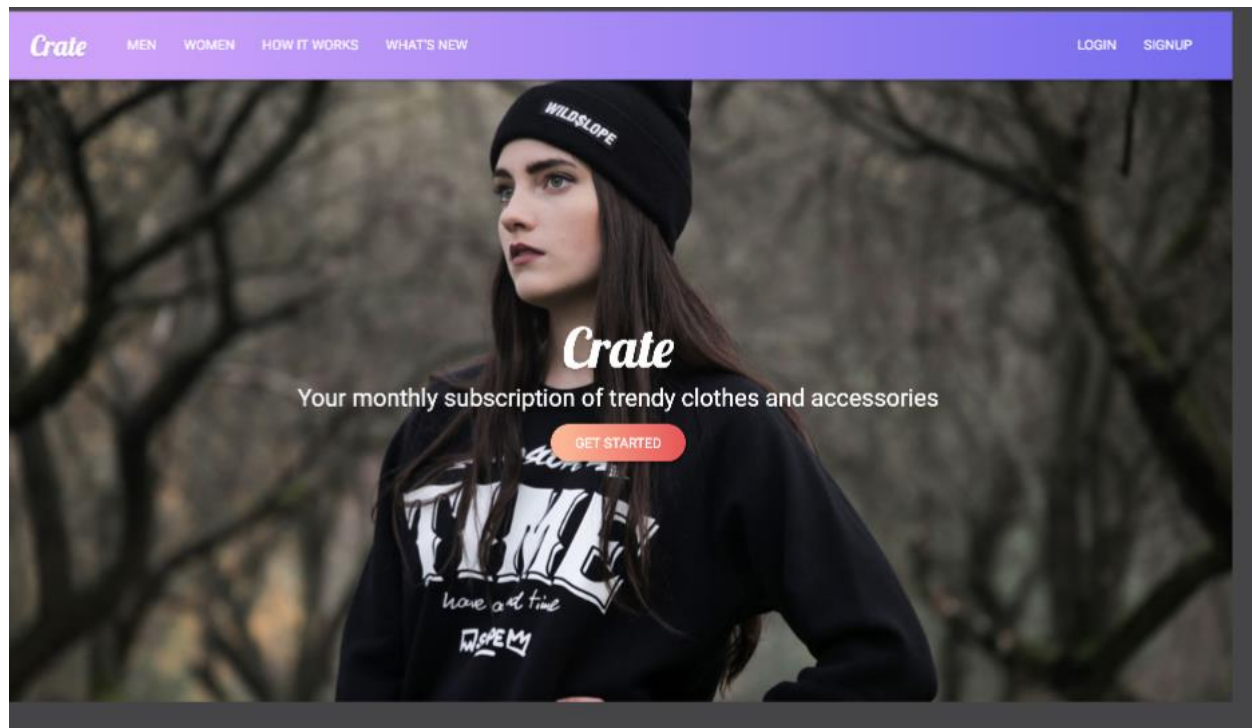
This repository is open for contributions. Developers can fork the repo, make their changes, and submit pull requests to contribute to the project. Issues and bugs can be reported through GitHub's Issues section.

License

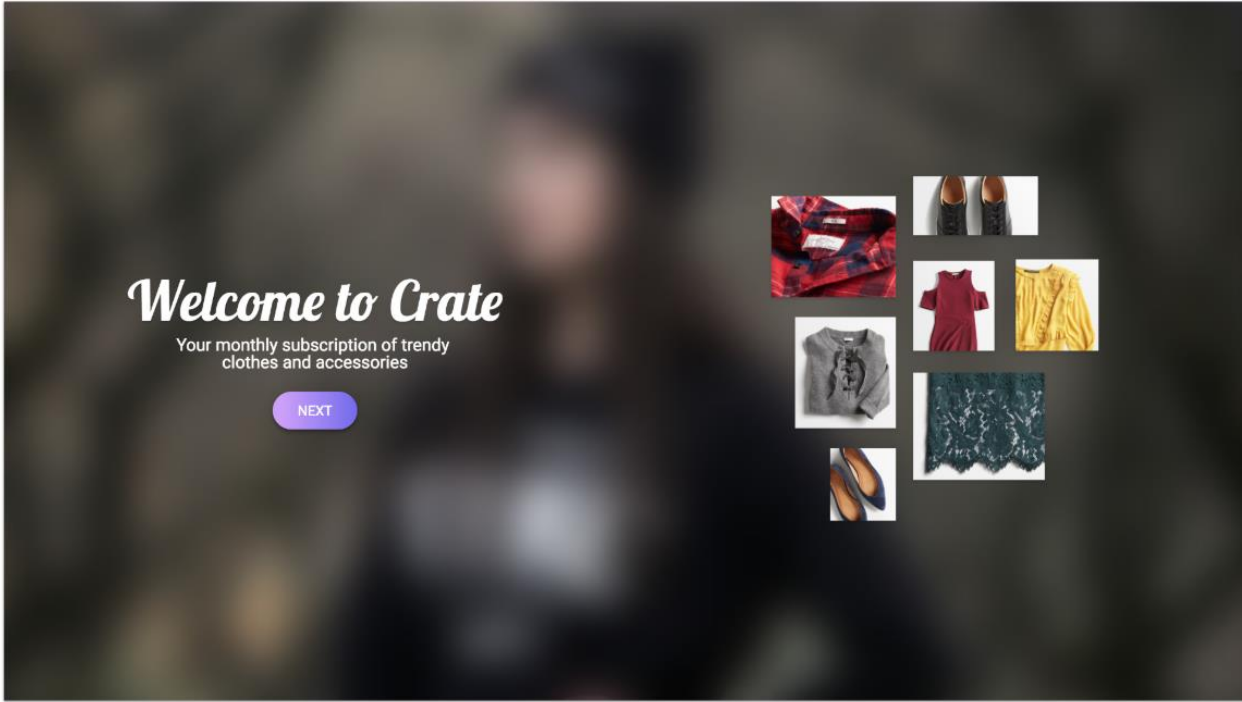
The repository is licensed under the **MIT License**, allowing you to freely use, modify, and distribute the code.

This repository provides a great starting point for developers looking to learn about full-stack development, **GraphQL** implementation, or working with modern technologies like **Sequelize**, **React**, and **React Native**. It's also a valuable resource for anyone interested in building scalable applications with a clear structure and well-organized codebase.

Output Screen Shots:



Tablet:

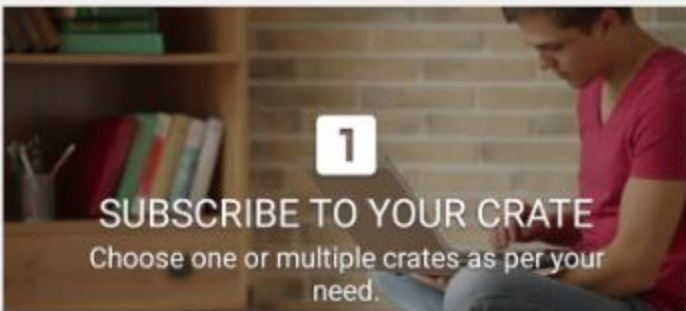


Mobile:



Your monthly subscription of trendy
clothes and accessories

+ GET SUBSCRIPTION



Home



Crates



What's New



Account